

What is Caching?

Caching is the process of storing copies of data or results of expensive operations (e.g., DB queries, API calls) in a **temporary storage layer** (cache), so future requests can be served **faster**.

- **Purpose:** Improve performance, reduce latency, decrease load on backend systems (like databases or APIs).
 - **Example:** Instead of querying the database each time for a product's details, you store the result in cache and serve from there until it becomes stale.
-

Types of Caching in General

1. **In-Memory Caching:** Cache is stored in the same memory space as your application (e.g., using HashMap, ConcurrentHashMap).
 2. **Distributed Caching:** Cache is stored in an external system accessible to multiple instances (e.g., Redis, Memcached).
-

What is Redis?

Redis (Remote Dictionary Server) is an open-source, in-memory **key-value data store**, often used as a **distributed cache**, message broker, or even lightweight database.

- **High performance:** Stores everything in memory.
 - **Supports TTL (Time to Live)** for cache expiry.
 - **Advanced data structures:** Lists, Sets, Sorted Sets, Hashes.
 - **Persistence options:** RDB snapshots, AOF logs.
-

Caching Before Redis in Spring Boot

Sure! Here's a detailed breakdown of caching before Redis in Spring Boot, the different approaches and annotations, and why Redis improves things:

Caching Before Redis in Spring Boot

Spring Boot has built-in support for caching using abstraction from spring-context module, even without Redis. It allows developers to annotate methods whose results should be cached to avoid repeated processing.

Step 1: Enable Caching

To use caching in Spring Boot, you must first **enable it**:

```
@SpringBootApplication
@EnableCaching

public class Application {

    public static void main(String[] args) {

        SpringApplication.run(Application.class, args);

    }

}
```

Step 2: Use Annotations to Implement Caching

Spring provides several **cache-related annotations**:

@Cacheable

Caches the result of a method.

```
@Cacheable(value = "users", key = "#userId")

public User getUserById(String userId) {

    return userRepository.findById(userId);

}
```

- **What it does:**
 - Checks if the result is already in the cache.
 - If yes, returns it.
 - If no, executes the method and stores the result in the cache.
-

@CachePut

Updates (or adds) a value in the cache **without skipping method execution**.

```
@CachePut(value = "users", key = "#user.id")

public User updateUser(User user) {

    return userRepository.save(user);

}
```

- **Use case:** When you want to update cache after a save/update operation.
-

@CacheEvict

Removes an entry from the cache.

```
@CacheEvict(value = "users", key = "#userId")

public void deleteUser(String userId) {

    userRepository.deleteById(userId);

}
```

- **Use case:** After delete operations, clear cache to avoid stale data.
-

@Caching

A container for combining multiple cache operations.

```
@Caching(

    evict = {

        @CacheEvict(value = "users", key = "#user.id"),

        @CacheEvict(value = "usersList", allEntries = true)

    }

)

public void deleteUser(User user) {

    userRepository.delete(user);

}
```

Default Caching Behavior (Without Redis)

If you don't configure a specific cache manager, Spring Boot uses:

SimpleCacheManager with ConcurrentHashMap

All cached values are stored **in-memory**, like a Map<String, Object>. This approach is:

Pros	Cons
Simple to set up	Not distributed
No external dependency	Memory-bound, not scalable
Fast lookup (in-JVM)	Loses data on restart
Good for development or single instance apps	No TTL support

Why Redis Makes Caching Better

When scaling your app, in-memory cache becomes a bottleneck. Redis solves these issues:

Benefits of Redis over In-Memory Caching

Feature	In-Memory (SimpleCacheManager)	Redis
Scope	JVM-local	Distributed (shared cache)
Scalability	Low	High
Data Persistence	No	Optional RDB/AOF

Feature	In-Memory (SimpleCacheManager)	Redis
TTL (Time-To-Live)	No (hard to configure)	Yes (native support)
Cluster-aware		
Handles app restart		
Memory Management	Uses app heap (GC pressure)	Uses Redis memory
Suitable for Microservices		
properties.yml		
redis:		
data:		
host: \${REDIS_HOST}		
port: 15641		
password: \${REDIS_PASSWORD}		
Absolutely! Here's a clean and structured Redis Notes (Spring Boot Edition) using		
your code as a reference:		

Redis Notes – Spring Boot Implementation (Custom Caching using RedisTemplate)

What is Redis?

- **Redis (Remote Dictionary Server)** is an in-memory key-value data store.
 - Commonly used for:
 - Caching
 - Session storage
 - Pub/Sub messaging
 - Offers **fast access**, **TTL support**, and **persistence** options.
-

Why Use Redis for Caching?

Problem with Traditional In-Memory Cache	Redis Advantage
Local to JVM instance (not shared)	Distributed & centralized
No TTL support	Native TTL support
Loses cache on app restart	Optional persistence (RDB/AOF)
Memory pressure on app	Redis manages its own memory
Not scalable in multi-instance setup	Scalable across services

Redis Integration in Spring Boot – Using RedisTemplate

Step 1: Add Dependency (Maven)

```
<dependency>  
  
<groupId>org.springframework.boot</groupId>  
  
<artifactId>spring-boot-starter-data-redis</artifactId>  
  
</dependency>
```

Step 2: Redis Configuration

@Configuration

```
public class RedisConfig {
```

@Bean

```
public RedisTemplate redisTemplate(RedisConnectionFactory factory){
```

```
    RedisTemplate redisTemplate = new RedisTemplate<>();
```

```
    redisTemplate.setConnectionFactory(factory);
```

```
    redisTemplate.setKeySerializer(new StringRedisSerializer());
```

```
    redisTemplate.setValueSerializer(new StringRedisSerializer());
```

```
    return redisTemplate;
```

```
}
```

```
}
```

Explanation:

- RedisTemplate is the core utility to interact with Redis.
- Key and value are serialized as plain strings using StringRedisSerializer.

RedisService – Custom Cache Wrapper

@Service

@Slf4j

```
public class RedisService {
```

@Autowired

```
    private RedisTemplate redisTemplate;
```

```
    public <T> T get(String key, Class<T> entityClass) {
```

```
        try {
```

```
            Object o = redisTemplate.opsForValue().get(key);
```

```
            ObjectMapper mapper = new ObjectMapper();
```

```

return mapper.readValue(o.toString(), entityClass);
} catch (Exception e) {
log.error("Exception ", e);
return null;
}
}

public void set(String key, Object o, Long ttl) {
try {
ObjectMapper objectMapper = new ObjectMapper();
String jsonValue = objectMapper.writeValueAsString(o);
redisTemplate.opsForValue().set(key, jsonValue, ttl, TimeUnit.SECONDS);
} catch (Exception e) {
log.error("Exception ", e);
}
}
}

```

Key Points:

- **get(...):** Fetches data from Redis, deserializes it into a Java object.
- **set(...):** Converts Java object to JSON string, sets it with **TTL (Time-To-Live)**.
- Uses Jackson's ObjectMapper for (de)serialization.

Example Use Case: Weather Service

@Autowired

private RedisService redisService;

public WeatherResponse getWeather(String city) {

WeatherResponse weatherResponse = redisService.get("weather_of_" + city,


```
WeatherResponse.class);

if (weatherResponse != null) {
    return weatherResponse; // Return from cache
} else {
    // Call external API if not cached

    String finalAPI = appCache.appCache.get(AppCache.keys.WEATHER_API.toString())
        .replace(Placeholders.CITY, city)
        .replace(Placeholders.API_KEY, apiKey);

    ResponseEntity<WeatherResponse> response = restTemplate.exchange(finalAPI,
        HttpMethod.POST, null, WeatherResponse.class);

    WeatherResponse body = response.getBody();

    if (body != null) {
        redisService.set("weather_of_" + city, body, 300L); // Cache for 5 minutes
    }

    return body;
}
}
```

Explanation:

- Key is dynamically generated as "weather_of_{city}".
- Checks Redis cache first.
- If not found, calls external weather API.
- Stores response in Redis with **5-minute TTL**.

Summary of Benefits (Custom Redis Caching with RedisTemplate)

Feature	Description
TTL Support	Custom expiry per entry (in seconds)
JSON Storage	Manual control using ObjectMapper
Key Naming	Fully customizable
Graceful Fallback	Fall back to API/database when cache misses
Logging	Easy to track caching logic and failures
Fine Control	You define serialization, deserialization, and cache lifecycle

When to Use This Approach vs @Cacheable

Use Case	RedisTemplate	@Cacheable
External API Responses	Best fit	Hard to manage
Full control over TTL		Limited customization
Complex key generation		Limited unless using SpEL
Simple method result caching	Overhead	Easier

Tips for Production

- Add cache invalidation or refresh logic (e.g., @Scheduled).
- Handle null values / empty responses.
- Avoid memory leaks by defining TTL always.
- Monitor Redis metrics (hit/miss, memory usage). Great! Below is a **detailed explanation** of the selected Redis topics tailored specifically for **interview preparation** — with examples, best practices, and potential questions:

3. Time-To-Live (TTL) Strategies in Redis

What is TTL?

TTL (Time-To-Live) is the **expiration time (in seconds)** after which a Redis key will automatically be deleted.

SET weather_of_pune "{...}" EX 300 # TTL of 300 seconds

Or programmatically:

```
redisTemplate.opsForValue().set("key", value, 300, TimeUnit.SECONDS);
```

Why TTL is Important:

- Prevents **stale data** from persisting.
 - Avoids **memory bloat** by automatically cleaning up old cache.
 - Helps manage **data freshness** in time-sensitive domains (e.g., weather, prices, sessions).
-

TTL Use Cases:

Use Case	Recommended TTL
Weather data	5–10 min
Product catalog	1–6 hours
Token/session	15–30 min
Flash sale inventory	Few seconds (high volatility)
Static reference data (e.g., countries)	No TTL or very long

Interview Tip:

What happens if I don't set TTL on cached entries? The cache will grow indefinitely and may cause **Out of Memory (OOM)** or **old/stale data issues**.

4. Cache Penetration, Breakdown, and Avalanche

a) Cache Penetration

- Repeated queries for **non-existent data** (e.g., invalid user ID).
- Always results in **DB hit**, never gets cached. **Solution:**
- Cache null/empty results temporarily (e.g., TTL 1 minute).
- Use a **Bloom Filter** to block invalid keys early.

```
if (result == null) {  
    redisTemplate.opsForValue().set("user_404", "NULL", 60, TimeUnit.SECONDS);  
}
```

b) Cache Breakdown (a.k.a. Cache Miss Stampede)

- A **hot key** expires, and many users hit the DB at once.
 - Common for frequently accessed but short-lived keys. **Solution:**
 - Use **mutex locks** (e.g., distributed lock) during cache rebuild.
 - **Pre-warm** the cache with background jobs.
 - Add **random TTL** to stagger expiration.
-

c) Cache Avalanche

- **Many keys expire at the same time**, leading to **DB overload**. **Solution:**
- Add **randomized TTL** (e.g., base TTL + random offset).
- Use different expiry windows for groups of keys.

```
long ttl = 300 + new Random().nextInt(60); // 5min ± 1min
```

Interview Tip:

How do you handle hot key expiration? Use a distributed lock to rebuild the cache once and let others wait or fallback to stale data temporarily.

7. Distributed Locking using Redis

Why Use Distributed Locks?

To coordinate access to shared resources **across multiple nodes**, e.g.:

- Prevent double booking
 - Avoid race conditions during cache refresh
 - Prevent duplicate processing of same event/job
-

Redis Lock via SETNX

```
Boolean acquired = redisTemplate.opsForValue().setIfAbsent("lock:product123",
"LOCKED", 10, TimeUnit.SECONDS);
if (Boolean.TRUE.equals(acquired)) {
    try {
        // critical section
    } finally {
        redisTemplate.delete("lock:product123");
    }
}
```

- **SETNX + TTL** ensures:
 - Lock is acquired if not present.
 - Automatically expires if not released (prevents deadlock).
-

Safe Lock Release (Compare-and-delete)

- Before deleting, check that the value belongs to you (to avoid deleting someone else's lock).
- Use a **UUID** to identify lock ownership.

Libraries for Robust Locking

- **Redisson** (highly recommended):
- Supports reentrant locks, semaphores, fair locks, etc.
- Example:

```
RLock lock = redissonClient.getLock("lock:order123");
lock.lock(10, TimeUnit.SECONDS);
try {
    // do something
} finally {
    lock.unlock();
}
```

Interview Tip:

How do you ensure only one thread processes a task across distributed systems? Use a Redis-based distributed lock with TTL and ownership check (or use Redisson for simplicity).

8. Redis Security & Hardening

Key Security Risks

Risk	Description
Open Redis port (6379)	Can be accessed without auth by attackers

Risk	Description
No password protection	Anyone on network can read/write
Exposed to public internet	Critical data can be dumped
No TLS	Data sent in plain text

Best Practices

1. **Require Authentication** In redis.conf:

requirepass YourSecurePassword

2. **Restrict Access**

- Bind to localhost or VPC IP only:

bind 127.0.0.1

- Use firewall or security groups.

3. **Enable TLS Encryption** Use stunnel or native TLS setup in Redis 6+.

4. **Avoid Wildcard Deletion** Dangerous commands like FLUSHALL or KEYS * should be disabled or restricted.

5. **Limit Command Set** (optional) Prevent abuse by disabling risky commands (e.g., CONFIG, DEBUG, SCRIPT).

6. **Key Naming Conventions** Use prefixes to avoid key collision across apps:

- "auth:user:{id}"
 - "weather:city:{name}"
-

Interview Tip:

How would you secure a Redis instance in production? Enable requirepass, bind to private IPs, use TLS, avoid exposing to internet, and implement proper key naming.